

Ejercicio Práctico: Sistema de Reserva de Entradas

1. Objetivo

Implementar un sistema simple de reservas utilizando Programación Orientada a Objetos y manejo de excepciones en Java.

El objetivo principal de esta actividad es que el alumno practique:

- Creación de clases y objetos.
- Relaciones entre clases.
- Herencia.
- Clases abstractas.
- Validación de datos.
- Manejo de excepciones.
- Delegación de excepciones mediante la cláusula throws.

2. Modelo del Sistema

El sistema estará compuesto por las siguientes clases:

Base (Clase Abstracta)

Evento

Reserva

Main

Relación entre las clases:

Un Evento puede tener múltiples Reservas.

Cada Reserva pertenece a un único Evento.

Tipo de relación:

Asociación Unidireccional 1 a N.

Evento 1 ----- * Reserva

3. Reglas de Negocio

Clase Base (Abstracta)

- El atributo id deberá autogenerarse secuencialmente mediante un contador estático.
- El atributo eliminado deberá inicializarse en false.
- El atributo createdAt deberá almacenar la fecha y hora de creación utilizando `LocalDateTime.now()`.
- Ninguna clase hija deberá recibir estos atributos por constructor.

Clase Evento

- Hereda de Base.
- Representa un evento para el cual pueden reservarse entradas.
- El método reservarLugares(Integer cantidad) deberá validar:
 - Que la cantidad sea mayor que cero.
 - Que exista capacidad disponible.
- Si alguna validación falla deberá lanzar una excepción utilizando throws.

Clase Reserva

- Hereda de Base.
- Representa la reserva realizada por un cliente.
- Mantiene una asociación con un objeto Evento.

Al construirse deberá validar:

- Que el evento no sea nulo.
- Que el nombre del cliente no esté vacío.
- Que la cantidad solicitada sea válida.

Si alguna validación falla deberá lanzar una excepción.

4. Uso Obligatorio de throws

Para esta actividad se deberá utilizar la cláusula throws para delegar las excepciones hacia los niveles superiores del sistema.

Ejemplo:

```
public void reservarLugares(Integer cantidad) throws Exception
```

Si el método detecta un error, deberá lanzar una excepción y no capturarla internamente.

Las excepciones deberán propagarse utilizando throws hasta llegar al método Main, donde serán manejadas mediante try-catch.

5. Pruebas a Realizar en Main

Caso 1: Reserva Exitosa

- Crear un Evento llamado 'Concierto de Rock' con capacidad para 10 personas.
- Realizar una reserva de 3 entradas.
- Mostrar la reserva y la capacidad restante.

Caso 2: Cantidad Inválida

- Intentar reservar 0 entradas.
- Capturar la excepción e imprimir el mensaje correspondiente.

Caso 3: Exceso de Capacidad

- Intentar reservar 20 entradas.
- Capturar la excepción e imprimir el mensaje correspondiente.

Caso 4: Cliente Inválido

- Intentar crear una reserva con nombreCliente vacío.
- Capturar la excepción e imprimir el mensaje correspondiente.

6. Recordatorio Importante

Cuando un método declara:

`throws Exception`

está indicando que puede generar una excepción y que no la manejará internamente.

Será responsabilidad del método que lo invoque decidir cómo tratar el error.

Por esta razón, las excepciones deberán capturarse únicamente en la clase Main mediante bloques try-catch.

7. Desafío Adicional

Crear las siguientes excepciones personalizadas:

- `CapacidadInsuficienteException`
- `ClienteInvalidoException`

Utilizar estas excepciones en lugar de la clase `Exception` genérica cuando corresponda.

De esta manera se podrán identificar con mayor precisión los distintos errores producidos por el sistema.